

Backend API & WebSocket Requirements

Clinic Management System — Billing & Token Queue Module

1. Overview

This document defines backend requirements for two modules:
the Billing module and the Real-Time Token Queue module.

System has four user roles:

Patient	Books appointments online. Views own bills and token status.
Doctor	Views all consultation and medication bills. Controls session timer.
Receptionist	Collects second-half consultation fee. Manages walk-in tokens.
Medical	Raises and views medication / pharmacy bills.

2. Module 1 — Billing

2.1 Consultation Fee Flow

The consultation fee is split into two parts:

Half Payment (50%) — Collected at the time of online appointment booking by the patient.

Remaining Half (50%) — Collected by the receptionist when the patient arrives at the clinic.

2.2 Bill Types

Consultation Bill	Covers the full consultation fee split into 2 payments. Visible to: Patient, Doctor, Receptionist.
Medication Bill	Raised by the Medical role for prescribed medicines or lab. Visible to: Patient, Doctor, Medical.

2.3 Role-wise Bill Visibility

Patient	Own consultation bill (both payment parts) + own medication bill.
Doctor	All consultation bills + all medication bills across all patients.
Receptionist	Consultation bills — specifically to collect the second-half payment.
Medical	Medication / pharmacy bills only.

2.4 APIs — Billing

Note: One GET endpoint serves all roles. Backend filters records based on the requesting user role.

POST /bills/consultation	Create a consultation bill at booking. Store 50% as paid (online), 50% as pending.
PATCH /bills/consultation/{bill_id}/collect-second-half	Receptionist marks the second half as collected.
POST /bills/medication	Medical role creates a medication bill linked to an appointment.
GET /bills	Fetch bills. Accepts query params: role, patient_id, appointment_id, type (consultation / medication). Backend filters by the authenticated user role automatically.

3. Module 2 — Real-Time Token Queue (WebSocket)

Everything in this module is handled over WebSocket. No polling.

All connected clients receive live updates automatically when queue or session state changes.

3.1 Doctor Session — Start / Stop

The Doctor login has two controls: Start Session and Stop Session.

Start Session: Doctor presses Start. Exact timestamp is recorded (e.g. 11:13:04) and broadcast to all connected clients. This signals the doctor is now ready to consult.

Stop Session: Session end time is recorded. All clients are notified the doctor is no longer consulting.

WebSocket event broadcast when session starts:

event	"session_started"
doctor_id	ID of the doctor
started_at	Exact timestamp e.g. "11:13:04"
status	"active"

3.2 Visit Type & Time Allocation

Each appointment has a visit type. Duration is used to predict queue timings.

First Visit	10 minutes
Follow-up	5 minutes
Acute	5 minutes
Critical	15 minutes

Note: These are estimates only. Actual time may vary. System adjusts in real time.

3.3 Token Assignment & Predicted Time Calculation

When a patient books and the session is active, they are assigned a token number.

Predicted time = Session Start Time + sum of durations of all tokens ahead in the queue.

Example:

Doctor session started at 11:13

Token 1 — First Visit (10 min) -> Predicted time: 11:13

Token 2 — Acute (5 min) -> Predicted time: 11:23

Token 3 — Follow-up (5 min) -> Predicted time: 11:28

3.4 Real-Time Queue Update Logic

When a doctor finishes with a patient earlier or later than estimated, all subsequent predicted times must be recalculated and broadcast to all clients via WebSocket immediately.

Trigger for recalculation:

Doctor marks current consultation complete (moves to next token).

The actual time the doctor moved to the next token is used as the new base.

WebSocket event broadcast on queue update:

event	"queue_updated"
updated_at	Timestamp of the update
queue	Array of all active tokens with: token_number, patient_id, visit_type, estimated_duration, predicted_time

3.5 APIs — Token Queue

Note: REST APIs are only for initial data load and token creation. WebSocket handles all real-time updates.

POST /appointments	Patient books appointment. Returns: appointment_id, token_number, predicted_time. Also triggers WebSocket broadcast to update all clients.
GET /queue/today	Returns full token queue for today including token numbers, visit types, and predicted times. Used on initial page load before WebSocket connects.
PATCH /queue/next	Doctor calls this to mark current consultation done and move to next token. Triggers server-side recalculation and WebSocket broadcast of updated predicted times.
PATCH /session	Doctor starts or stops the session. Body: { action: start or stop }. Triggers WebSocket broadcast.

3.6 WebSocket Events Summary

session_started	Server to All Clients. Doctor started session. Includes started_at timestamp.
session_stopped	Server to All Clients. Doctor ended the session.
queue_updated	Server to All Clients. Queue recalculated. Full updated token list with new predicted times.
token_assigned	Server to All Clients. New token created. Clients update their local queue display.

4. API Summary

Billing APIs

POST /bills/consultation	Create consultation bill (50% paid, 50% pending)
PATCH /bills/consultation/{id}/collect-second-half	Receptionist collects second half
POST /bills/medication	Create medication bill
GET /bills	Fetch bills, role-filtered via query params

Token Queue APIs

POST /appointments	Book appointment, assign token, return predicted time
GET /queue/today	Load today's full queue on page init
PATCH /queue/next	Move to next token, trigger recalculation + WebSocket broadcast
PATCH /session	Start or stop doctor session, trigger WebSocket broadcast

Total: 8 REST endpoints + 1 WebSocket connection

All real-time updates are handled exclusively via WebSocket. No polling required.

All endpoints must validate the authenticated user role and restrict access accordingly.

Predicted times must be recalculated server-side and pushed via WebSocket.

Clients should never compute timings locally.